

Real-Time Collaboration Platform

Enterprise-Grade Multi-User Document Collaboration System

Prepared for: LogicVeda - Web Development Domain

Project Code: lv1-2026-03-01

Prepared by: Shivam

Handle: @harsadash

Date: May 18, 2026

Version: Final Submission Draft

Live Demo: <https://editorial.justshivamm.in>

Backend Health: <https://realtime-collab-backend-oiou.onrender.com/healthz>

GitHub Repository: https://github.com/shivam2931120/realtime_collab

Demo Video: Add the final 3-7 minute Loom/YouTube/Drive link here after recording.

1. Executive Summary

The Real-Time Collaboration Platform, branded as Editorial in the deployed application, is a production-ready full-stack collaborative document editor inspired by tools such as Google Docs, Etherpad, Notion, and Coda. It enables users to create, edit, share, comment on, search, organize, import, export, and collaborate on rich text documents in real time.

The platform focuses on a practical collaboration workflow for remote teams, student groups, writers, consultants, and small product teams. It reduces version confusion by keeping documents in a single shared workspace and provides immediate feedback through live editing, active-user presence, comments, notifications, and email alerts.

The implementation includes a React 19 + TypeScript frontend, Node.js 22 + Express backend, Socket.io realtime communication, Supabase persistence, SMTP email delivery, production deployment on Vercel and Render, and documentation for setup, architecture, secrets, SMTP, and deployment operations.

2. Project Overview

2.1 Vision

The project aims to deliver a high-fidelity collaborative document workspace where multiple users can work on shared content with visible realtime presence, access controls, and communication features. The goal is to show end-to-end ownership of a modern full-stack system: frontend UX, backend APIs, realtime transport, persistence, authentication, notifications, deployment, and operational readiness.

2.2 Objectives

- Build a usable Google Docs style document editor.
- Support document creation, listing, reading, updating, deleting, and sharing.
- Enable realtime editing and visible collaborator presence.
- Provide role-based access control for owners, editors, and viewers.
- Add comments, resolved/open states, anchored comment references, and mentions.
- Send email notifications for document sharing, mentions, and password reset.
- Provide version history with manual snapshots, automatic snapshots, and restore.
- Provide search, folders, tags, templates, analytics, import, and export.
- Deploy the application publicly with HTTPS.
- Document setup, architecture, verification, and operations.

2.3 Target Users

- Remote software teams working on specs, planning documents, and notes.
- University project groups collaborating on reports and assignments.
- Content creators and writers drafting shared articles or scripts.
- Freelancers and consultants collaborating with clients.
- Startup teams building lightweight internal knowledge bases.

2.4 Business Value

- Reduces email-based document back-and-forth.
- Avoids duplicate versions of the same file across users.
- Provides instant visibility into who is currently working.
- Enables structured feedback through comments and mentions.
- Allows controlled collaboration through role-based permissions.
- Supports export formats for handoff and submission.

2.5 Non-Functional Goals

Goal	Target	Implementation Notes
Latency	Less than 200 ms	target for edit propagation in normal conditions Socket.io room broadcasts are used for low-latency update delivery.
Concurrency	Designed for many active users per document	Socket rooms isolate document traffic; Redis support is included for future multi-instance scaling.
Availability	Public HTTPS deployment	Frontend is deployed on Vercel, backend on Render.
Security	Secure defaults and OWASP awareness	CORS allowlist, auth middleware, rate limiting, security headers, no committed secrets, and token validation are used.
Maintainability	Clear repo structure	Separate `frontend/`, `backend/`, `docs/`, `postman/`, deployment configs, and schema file.

3. Key Features and Acceptance Criteria

ID	Feature	Description	Acceptance Criteria / Status
F01	User Authentication	Users can register and sign in with email/password. The system supports access tokens, refresh tokens, logout, password reset, and a demo access flow.	Implemented through `api/auth/register`, `api/auth/login`, `api/auth/refresh`, `api/auth/logout`, `api/auth/password-reset/request`, `api/auth/password-reset/confirm`, and `api/auth/session`.
F02	Document CRUD	Users can create, list, open, update, and delete documents.	Implemented. Owner-only delete is enforced. Soft delete is supported through `deleted_at` when the latest schema is applied, with compatibility fallback for older deployed schemas.
F03	Real-Time Editing	Multiple users can edit a shared document and see changes propagate through sockets.	Implemented using Socket.io document rooms. Smoke tests verified two active sessions, realtime propagation, and persisted content.
F04	User Presence	Active users are visible while viewing/editing a document.	Implemented with `join-doc`, `active-users`, `cursor-move`, and `presence-ping` events. The UI shows online collaborators, idle status, and cursor labels.
F05	Inline Comments and Mentions	Users can add comments, resolve/reopen comments, anchor comments to selected text, and mention users by email.	Implemented. Comments store position metadata and `@email` mentions generate notifications and email.
F06	Version History	Users can manually save snapshots and restore previous	

versions. | Implemented. The backend also creates automatic snapshots during active edits. |

| F07 | Role-Based Access Control | Documents support owner, editor, and viewer roles. | Implemented across REST APIs and Socket.io editing. Viewers cannot modify content or tags. Only owners can update sharing and delete documents. |

| F08 | Notifications | Users receive in-app and email notifications for sharing and mentions. | Implemented. SMTP was verified successfully. Notifications can be marked read individually or all at once. |

| F09 | Search and Filtering | Users can search documents by title, owner, content, and tags. | Implemented in dashboard and `/api/docs/search`. Folders, tags, templates, and analytics are also included. |

| F10 | Offline Awareness | Users are warned when realtime or browser connectivity is unavailable. | Implemented. The editor shows offline/realtime banners and queues saves for retry. |

4. Technology Stack

Category	Technology	Rationale
Frontend Framework	React 19, Vite, TypeScript	Fast development, modular UI, strong typing, production build support.
Styling	Tailwind CSS, Material UI/Joy UI components, Material Symbols	Responsive layout, consistent controls, and fast UI iteration.
Rich Text Editor	Tiptap / ProseMirror	Extensible rich editor with tables, tasks, links, highlights, images, YouTube embeds, slash commands, and formatting controls.
State Management	Zustand	Lightweight global stores for auth, documents, UI, and preferences.
HTTP Client	Axios	API abstraction with auth token interceptor.
Real-Time Transport	Socket.io	WebSocket-based realtime rooms with fallback support and simple event model.
Backend	Node.js 22, Express, TypeScript	Stable production runtime, clear route/controller structure, strong typing.
Database	Supabase PostgreSQL	Managed persistence for documents, collaborators, comments, notifications, versions, folders, tags, templates, and analytics events.
Email	Nodemailer + SMTP	Transactional share, mention, and password reset emails.
Export Tools	PDFKit, docx, Turndown, html-to-text, Mammoth	Export/import support for PDF, DOCX, Markdown, HTML, and text workflows.
Optional Cache/PubSub	Redis / ioredis	Included for future scaling, cache invalidation, and pub/sub support.
Deployment	Vercel + Render	Vercel serves the frontend; Render hosts the backend API and Socket.io server.
Documentation	Markdown docs, Postman collection, schema SQL	Clear local setup, deployment, architecture, SMTP, release, and security notes.

5. Architecture

5.1 High-Level Architecture

```

```text
Users / Browsers
 |
 | HTTPS
 v
Vercel Frontend
React + Vite + TypeScript
 |
 | REST API + Socket.io

```

```

 v
Render Backend
Node.js + Express + Socket.io
 |
 | Supabase client
 v
Supabase PostgreSQL
Documents, collaborators, comments,
versions, folders, tags, notifications,
templates, analytics events
 |
 | SMTP
 v
Email Provider
Share, mention, password reset emails
```

```

5.2 Main Components

- Frontend application: authentication screens, dashboard, workspace layout, rich editor, comments panel, version history panel, notifications menu, analytics page, template library, search and folders.
- Backend API: auth routes, document routes, comments, folders, versions, discovery/search, tags, templates, imports/exports, notifications, analytics, health and metrics.
- Realtime layer: Socket.io rooms per document with authenticated socket handshake, active user tracking, cursor movement, presence pings, document update broadcasts, and room cleanup on leave/disconnect.
- Database layer: Supabase stores persistent entities and relationships using the schema in `supabase\_schema.sql`.
- Email layer: Nodemailer sends SMTP-backed document share, mention, and password reset emails.

5.3 Data Model Summary

| Entity | Purpose |
|------------------------------|---|
| --- --- | |
| `auth_users` | Password-auth users when the latest schema is applied. |
| `auth_refresh_tokens` | Refresh token records for session rotation. |
| `auth_password_reset_tokens` | Password reset token records. |
| `documents` | Main document records with title, content, owner, folder, timestamps, and optional soft delete. |
| `document_collaborators` | Per-document user roles: editor or viewer. Owner is stored on the document itself. |
| `comments` | Document comments, resolved state, author, and optional anchored position metadata. |
| `notifications` | In-app notification records for shares and mentions. |
| `document_versions` | Manual and automatic content snapshots. |
| `folders` | Workspace folder hierarchy. |
| `document_tags` | Tags attached to documents. |
| `document_templates` | User and system templates. |
| `document_events` | Analytics events for views, edits, shares, comments, imports, exports, and document creation. |

6. Functional Walkthrough

6.1 Authentication

The platform supports two authentication paths:

1. Production email/password flow:
 - Register with email and password.

- Login with email and password.
- Receive an access token and refresh token.
- Refresh the session when the access token expires.
- Logout by revoking the refresh token.
- Request and confirm password reset through email.

2. Demo access flow:

- The login screen includes a "Continue with demo" option.
- This creates a signed session token for evaluator access without account setup.

This satisfies the live demo requirement that core functionality should be available without mandatory signup friction.

6.2 Dashboard and Workspace

The dashboard lists recent documents and folders. Users can:

- Create new documents.
- Create folders.
- Move documents and folders through drag-and-drop.
- Search workspace content.
- Pin and favorite documents locally.
- Delete documents as owner.
- Open shared documents based on role.

6.3 Rich Text Editor

The editor includes a document canvas and toolbar with:

- Headings and paragraph styles.
- Bold, italic, underline, strike, inline code.
- Bullet lists, numbered lists, task lists.
- Links, images, YouTube embeds.
- Tables with row/column controls.
- Code blocks and horizontal rules.
- Text color and highlight colors.
- Alignment controls.
- Find and replace.
- Selection transforms: uppercase, lowercase, title case, timestamp.
- Import and export menus.

6.4 Real-Time Collaboration

When users open the same document, the backend places each authenticated socket in a document-specific room. Editor updates are emitted through `send-changes`, persisted in Supabase, and broadcast to other active users through `receive-changes`.

Realtime presence is visible through:

- Active collaborator avatars.
- Online/idle state.
- Last seen labels.
- Remote cursor markers with user labels.
- Socket connection state in the editor header.

Production smoke testing verified:

- Two active Socket.io sessions joined the same document.
- Presence reached 2 active users.
- A collaborator edit propagated to the other session.
- The new content was persisted and readable from the REST API.

6.5 Sharing and Permissions

Owners can share documents by email and assign:

- Editor: can edit document content and tags.
- Viewer: can open and read but cannot edit.
- Owner: full control, including sharing and deletion.

Permissions are enforced in both REST endpoints and Socket.io events so a viewer cannot bypass the UI by sending socket updates.

6.6 Comments, Mentions, and Notifications

Users can add comments from the editor sidebar. Comments include:

- Author email.
- Created timestamp.
- Resolved/open state.
- Optional anchor position for selected text.
- Linked text preview for anchored comments.

Mentioning an email in the format `@user@example.com` creates:

- In-app notification.
- Email notification through SMTP when credentials are configured.

6.7 Version History

Version history supports:

- Manual snapshots from the editor.
- Automatic snapshots during active editing.
- Timeline list of snapshots.
- Restore endpoint that persists the restored content.

This gives users a recovery path if edits need to be rolled back.

6.8 Search, Tags, Templates, Import, Export, and Analytics

Additional productivity features include:

- Search by title, content, owner, and tags.
- Popular tags.
- Per-document tags.
- Template library with default templates and custom templates.
- Import from text, Markdown, HTML, and DOCX.
- Export to Markdown, HTML, TXT, PDF, and DOCX.
- Analytics summary for document activity, views, edits, shares, imports, exports, comments, and versions.

7. Security and OWASP Awareness

7.1 Authentication Security

- Access tokens are HMAC signed.
- Refresh tokens are rotated when the schema-backed token table is available.
- Passwords are hashed using Node.js `crypto.scrypt`.
- Password reset tokens expire after 30 minutes.
- Password reset request responses do not reveal whether an email exists.

7.2 Authorization

- Every protected REST route uses auth middleware.
- Document access checks verify owner/collaborator membership.
- Owner-only operations include delete and sharing updates.
- Viewer restrictions are enforced server-side.
- Socket events validate document access and role before joining or editing.

7.3 API Hardening

- CORS is restricted to configured frontend origins.
- Express `x-powered-by` header is disabled.
- Security headers include:
 - `X-Content-Type-Options: nosniff`
 - `X-Frame-Options: DENY`
 - `Referrer-Policy: no-referrer`
 - `Permissions-Policy`
 - `Cross-Origin-Resource-Policy`
- Auth routes have rate limiting.
- JSON body size is limited.

7.4 Secrets Management

- `.env` files are ignored from Git.
- `.env.example` documents required variables without secrets.
- Production secrets are stored in deployment provider environment variables.
- Secret rotation guidance is documented in `docs/SECRETS\_ROTATION.md`.

8. Performance and Scalability

8.1 Frontend Performance

- Vite production bundling is used.
- React routes are lazy-loaded with `Suspense`.
- Editor and workspace pages are split into separate chunks.
- The UI uses optimized stores through Zustand.

8.2 Backend Performance

- Socket.io rooms isolate realtime updates per document.
- Search and tag endpoints support Redis cache helpers when Redis is configured.
- Analytics reads are scoped to accessible document IDs.
- Health and metrics endpoints allow monitoring.

8.3 Scaling Strategy

The current deployment is suitable for a single backend instance. To scale horizontally:

- Add Redis.
- Configure Socket.io Redis adapter.
- Keep sticky sessions enabled if required by the hosting platform.
- Move heavier email jobs into a background queue.
- Add load testing for 50 concurrent editors per document.

9. Deployment and Operations

9.1 Production URLs

- Frontend: <https://editorial.justshivamm.in>
- Vercel deployment: <https://realtime-collab-kmo4nvirs.vercel.app>
- Backend: <https://realtime-collab-backend-oiou.onrender.com>

- Backend health: <https://realtime-collab-backend-oiou.onrender.com/healthz>
- Backend API health: <https://realtime-collab-backend-oiou.onrender.com/api/health>
- Backend metrics: <https://realtime-collab-backend-oiou.onrender.com/metrics>

9.2 Hosting

| Layer | Platform | Details |
|----------|----------------------------------|--|
| Frontend | Vercel | Builds the Vite React app from `frontend/` and serves `frontend/dist`. |
| Backend | Render | Runs the Node.js Express + Socket.io backend from `backend/`. |
| Database | Supabase | Stores documents and collaboration data. |
| Email | SMTP provider through Nodemailer | Sends transactional emails. |

9.3 Environment Variables

Backend:

```
```env
PORT=5000
CLIENT_URL=https://editorial.justshivamm.in
CLIENT_URLS=
NODE_ENV=production
AUTH_TOKEN_SECRET=<secret>
AUTH_TOKEN_TTL_SECONDS=1209600
SUPABASE_URL=<supabase-url>
SUPABASE_SERVICE_KEY=<supabase-service-role-key>
SMTP_SERVICE=gmail
SMTP_HOST=smtp.gmail.com
SMTP_PORT=465
SMTP_USER=<smtp-user>
SMTP_PASS=<smtp-app-password>
SMTP_FROM=<from-address>
REDIS_URL=
```
```

Frontend:

```
```env
VITE_BACKEND_URL=https://realtime-collab-backend-oiou.onrender.com
VITE_API_URL=https://realtime-collab-backend-oiou.onrender.com/api
VITE_SOCKET_URL=https://realtime-collab-backend-oiou.onrender.com
```
```

9.4 Health and Monitoring

The backend exposes:

- `/healthz` for platform health checks.
- `/api/health` for API health checks.
- `/metrics` for Prometheus-style metrics.

Example metric:

```
```text
realtime_collab_uptime_seconds
realtime_collab_auth_rate_buckets
```
```

10. Testing and Verification

10.1 Build Verification

Backend:

```
```bash
cd backend
npm run build
```
```

Result: Passed.

Frontend:

```
```bash
cd frontend
npm run build
```
```

Result: Passed.

10.2 Local Smoke Tests

Command:

```
```bash
./scripts/smoke_check.sh http://localhost:5000
```
```

Verified:

- `/healthz`
- `/api/health`
- `/api/auth/session`

Result: Passed.

10.3 SMTP Verification

Command:

```
```bash
cd backend
npm run smtp:verify
```
```

Result: SMTP verify passed.

10.4 Realtime Verification

Local and production socket tests verified:

- Two authenticated sessions can connect to Socket.io.
- Both sessions can join the same document.
- Active-user presence reaches 2 users.
- Edits from one session are received by the other session.
- Updated document content persists in Supabase and is readable through REST.

Production verification result:

```
```json
{
 "ok": true,
 "activeUsers": 2,
}
```

```
"auth": "register-login-refresh-logout",
"realtime": "socket-and-persisted"
}
...
```

#### ### 10.5 Deployment Verification

- Vercel production deployment status: Ready.
- Frontend public URL returned HTML successfully.
- Render backend health endpoint returned OK.
- Render metrics endpoint returned metrics successfully.

---

#### ## 11. Visual Evidence to Include in the PDF

Add these 8 screenshots to the final PDF export:

1. Login/register screen with demo access button.
2. Dashboard showing recent documents and the new document modal.
3. Folder/workspace view showing document cards and search.
4. Editor with rich text toolbar and document canvas.
5. Two browser windows/tabs open on the same document showing realtime presence.
6. Remote cursor or active collaborators panel.
7. Comments sidebar showing an anchored comment and resolved/open controls.
8. Version history sidebar showing snapshots and restore option.

Recommended captions:

- "Authentication and evaluator-friendly demo access"
- "Workspace dashboard with document creation and sharing"
- "Rich text editing canvas with formatting controls"
- "Realtime collaboration with active users visible"
- "Anchored comments with mention-ready discussion"
- "Version history with snapshot restore"
- "In-app notifications for collaboration events"
- "Export options for PDF, DOCX, Markdown, HTML, and text"

---

#### ## 12. Challenges and Solutions

##### ### Challenge 1: Realtime editing and persistence

Realtime collaboration must feel instant while also keeping the database updated. The solution uses Socket.io rooms for immediate propagation and Supabase updates for persistence. Save retries and offline warnings reduce the chance of silent data loss.

##### ### Challenge 2: Access control across REST and sockets

Restricting only the UI is not enough because users could still call APIs or emit socket events manually. The backend checks permissions for REST endpoints and socket events, including document join, content updates, cursor updates, comments, tags, sharing, and deletes.

##### ### Challenge 3: Email delivery

SMTP configuration often fails because of incorrect credentials or provider restrictions. A dedicated `smtp:verify` script was added so email readiness can be tested independently before running collaboration flows.

##### ### Challenge 4: Schema compatibility

The latest schema includes password-auth tables and soft-delete columns. To keep the live demo working before every migration is applied, the backend includes compatibility behavior for older deployed schemas while still supporting the preferred schema-backed implementation.

---

### ## 13. Future Roadmap

- Add Yjs or another CRDT engine for stronger concurrent text conflict handling.
- Add Socket.io Redis adapter for multi-instance horizontal scaling.
- Add Playwright end-to-end tests for browser-based multi-user collaboration.
- Add load testing with Artillery for 50 concurrent editors.
- Add CI matrix for lint, test, build, and deployment checks.
- Add Kubernetes manifests and HPA for container orchestration.
- Add Prometheus/Grafana dashboards for production monitoring.
- Add real screenshot assets and demo GIFs inside the README.
- Add document-level public link sharing with expiring invite tokens.
- Add threaded replies under comments.

---

### ## 14. Personal Reflection

This project strengthened my understanding of full-stack product engineering beyond basic CRUD. The most valuable learning was connecting several production concerns into one coherent workflow: authentication, permissions, realtime socket events, persistent storage, email notifications, deployment, environment variables, and verification.

The project also showed that realtime systems need careful server-side checks. Even when the frontend disables editing for viewers, the backend still has to enforce the same rule for REST and Socket.io events. I also learned the importance of health checks, SMTP verification, deployment-specific environment variables, and documenting operational steps clearly.

The final result is a usable collaborative editor with enough depth to demonstrate modern frontend engineering, backend API design, realtime communication, access control, and deployment readiness.

---

### ## 15. Final Submission Deliverables

Deliverable	Required	Submission Value
Project Documentation / Report PDF	Yes	Export this report as `Shivam_Project1_RealTimeCollab_LogicVeda_March2026.pdf`.
Live Public Demo URL	Yes	<a href="https://editorial.justshivamm.in">https://editorial.justshivamm.in</a>
GitHub Repository	Yes	<a href="https://github.com/shivam2931120/realtime_collab">https://github.com/shivam2931120/realtime_collab</a>
README.md	Yes	Present at repo root: `README.md`.
Demo Video	Yes	Record a 3-7 minute walkthrough and paste the final URL here.

#### ### Recommended Demo Video Script

1. Open the live demo URL.
2. Sign in with demo access or register a new account.
3. Create a new document from the dashboard.
4. Share it with another email as editor.
5. Open the same document in a second browser/incognito window.
6. Type in one browser and show the update appearing in the other.
7. Show active users and cursor/presence indicators.
8. Add a comment and mention an email.

9. Create a version snapshot and restore it.
10. Export the document as PDF or DOCX.
11. Show the GitHub repository and README briefly.
12. End with deployment URLs and a summary of completed features.

#### ### Final Submission Checklist

- [ ] Export this report to PDF.
- [ ] Insert 5-10 screenshots into the Visual Evidence section.
- [ ] Upload the 3-7 minute demo video.
- [ ] Add the demo video URL to the report.
- [ ] Submit the live demo URL: <https://editorial.justshivamm.in>
- [ ] Submit the GitHub repository:  
[https://github.com/shivam2931120/realtime\\_collab](https://github.com/shivam2931120/realtime_collab)
- [ ] Confirm README is visible in the repository.
- [ ] Confirm the deployed app opens on mobile and desktop.
- [ ] Confirm demo access works before submission.

---

#### ## Appendix A: Important API Endpoints

##### Authentication:

- `POST /api/auth/session`
- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/refresh`
- `POST /api/auth/logout`
- `POST /api/auth/password-reset/request`
- `POST /api/auth/password-reset/confirm`
- `GET /api/auth/me`

##### Documents:

- `POST /api/docs`
- `GET /api/docs`
- `GET /api/docs/:id`
- `PUT /api/docs/:id`
- `DELETE /api/docs/:id`

##### Comments:

- `GET /api/docs/:id/comments`
- `POST /api/docs/:id/comments`
- `PUT /api/docs/:id/comments/:commentId`

##### Versions:

- `POST /api/docs/:id/versions`
- `GET /api/docs/:id/versions`
- `POST /api/docs/:id/versions/:versionId/restore`

##### Discovery and productivity:

- `GET /api/docs/search`
- `GET /api/docs/tags`
- `GET /api/docs/:id/tags`
- `PUT /api/docs/:id/tags`
- `GET /api/docs/templates`
- `POST /api/docs/templates`
- `POST /api/docs/templates/:templateId/apply`
- `POST /api/docs/import`
- `GET /api/docs/:id/export?format=markdown|html|txt|pdf|docx`

- `GET /api/docs/analytics?days=30`

#### Notifications:

- `GET /api/notifications`  
- `PUT /api/notifications/read-all`  
- `PUT /api/notifications/:id/read`

#### Operations:

- `GET /healthz`  
- `GET /api/health`  
- `GET /metrics`

#### ## Appendix B: Realtime Socket Events

- `join-doc`  
- `leave-doc`  
- `send-changes`  
- `receive-changes`  
- `cursor-move`  
- `active-users`  
- `presence-ping`  
- `doc-error`

#### ## Appendix C: Repository Structure

```
```text
realtime-collab/
  backend/
    src/
      controllers/
      middleware/
      routes/
      sockets/
      utils/
      scripts/
      package.json
  frontend/
    src/
      components/
      pages/
      services/
      store/
      package.json
  docs/
    ARCHITECTURE.md
    DEPLOYMENT.md
    SMTP.md
    SECRETS_ROTATION.md
    REQUIREMENTS_TRACEABILITY.md
    FINAL_REPORT_CONTENT.md
  postman/
  supabase_schema.sql
  render.yaml
  vercel.json
  README.md
  project.pdf
```
```